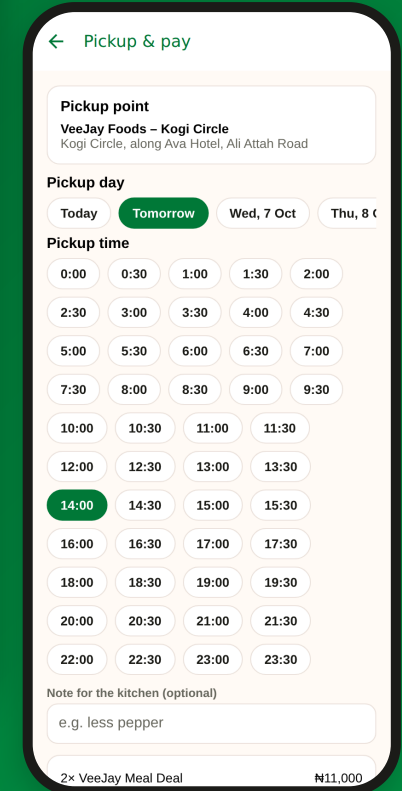
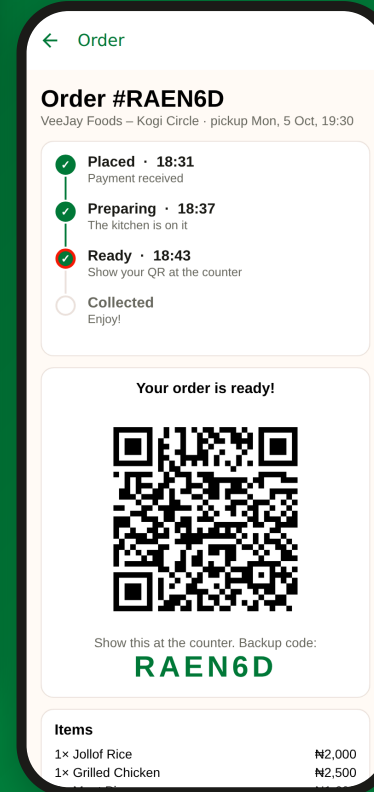
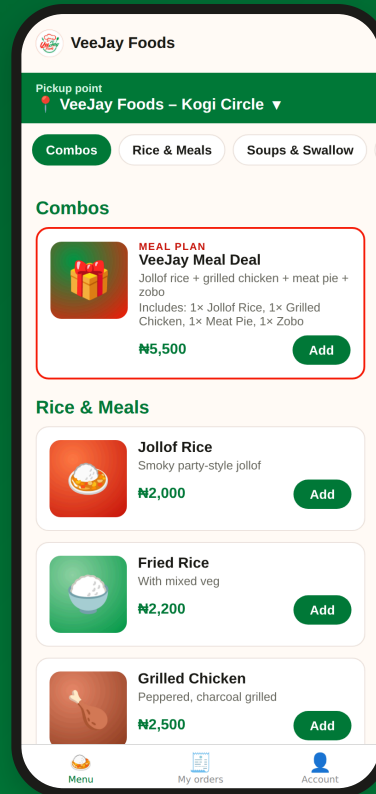




PRODUCT DESIGN & DELIVERY PLAN · V1.1

VeeJay Foods

Order ahead. Skip the queue. A mobile pre-order and pickup app for VeeJay Foods, Lokoja — pay with card, bank transfer or USSD, track your order, and collect with a QR code.



Prepared 5 October 2026 · VeeJay Foods · Kogi Circle, along Ava Hotel, Ali Attah Road, Lokoja, Kogi State · Open 24 hours

Customer app (Expo / React Native) · API (Node, Express, Prisma, PostgreSQL) · Staff & admin dashboard (React)

All screenshots are captured from the real code running against the real API. Menu items, prices and customers shown are sample data.

The problem

VeeJay Foods serves homemade African meals, pastries and local cuisine around the clock from Kogi Circle in Lokoja. Like any popular outlet it loses sales and goodwill at peak hours: long counter queues, orders shouted over a crowd, cash handling, and “where is my order?” with no answer. Customers abandon the queue; staff get overwhelmed in bursts and idle in between.

The idea

Let customers choose a **pickup point and a time slot, pay in advance**, and walk in to **scan a QR code**. The kitchen sees a live, time-ordered queue, so demand is spread across slots instead of arriving in a rush. Money is settled before food is made, and every order has an auditable trail.

One-line promise

“Pick it, pay it, collect it — no queue.”

Why it works for each side

Customer

- No waiting in line
- Choose the exact pickup time
- Pay by card, transfer or USSD
- Live status + push notifications
- Wrong order? Report with a photo, get refunded

Kitchen / counter

- Orders arrive already paid
- Slot capacity caps the rush
- Clear New → Preparing → Ready board
- Scan-to-collect, no disputes
- Works for day and night shifts

Owner

- Sales by day, outlet and item
- Sold-out toggles per outlet
- Refund control, audit trail
- Menu, prices and hours without a developer

Proposed success metrics

- Median counter wait at pickup < 2 min
- Cart → paid conversion
- Refund rate < 2% of orders
- Repeat orders within 14 days

Targets to agree with the business — none measured yet.



Amina — the customer

27, works near Kogi Circle in Lokoja. Has 25 minutes for lunch, hates queues, pays by bank transfer or card on her phone. Patchy mobile data on the commute.

Needs: fast reorder, certainty about pickup time, proof of payment, a way to fix mistakes.



Musa — counter staff

Runs the counter during the lunch rush, or on the night shift. Works on a shared tablet or phone, no time for menus.

Needs: one screen showing what to cook next, big buttons, a scanner that just works, a fallback when a phone dies.



The owner / admin

Wants to change prices, hide a sold-out dish, add a future branch, and see yesterday's sales over breakfast.

Needs: control without a developer, trustworthy numbers, safe refunds.

The core journey (customer)

1

Sign in

Phone number + SMS code

2

Browse

Menu, combos, per-outlet stock

3

Cart

Quantities, running total

4

Pick slot

Outlet, day, time

5

Pay

Paystack: card · transfer · USSD

6

Track

Placed → Preparing → Ready

7

Collect

Show QR, staff scan

If something goes wrong

- **Paid but screen didn't update** — the app asks the server to re-check with Paystack; the webhook is the safety net.
- **Phone dies at the counter** — staff type the 6-character order code instead of scanning.
- **Wrong or cold food** — report in-app with a photo; admin approves and the refund goes back to the original payment method.

Design rules that follow from the users

- A user with a paid, uncollected order is **never signed out** — they always have their QR.
- Every critical button is ≥ 50 px tall; the counter UI works one-handed.
- Prices are integers in kobo end-to-end — no rounding surprises in ₦.
- Network failures never look like “logged out” or “payment failed”.

Nine capabilities, all built

#	FEATURE	WHAT IT DOES	HOW IT WAS VERIFIED
1	Phone + OTP sign-in	6-digit SMS code (hashed, 5-min expiry, 5 attempts, rate-limited). 15-min access token + rotating 30-day refresh token. Never ends a session while an order is active.	16 automated tests incl. expiry, lock-out, replay, active-order rule
2	Menu & combos	Categories, items, combos / meal plans with components, images, prices in ₺, per-outlet “sold out” switches.	Browser-verified (app + dashboard)
3	Pickup points & slots	Outlets with weekly hours (including “24 hours”), prep time, slot length and per-slot capacity; capacity enforced under concurrent bookings. Ready for more branches.	4 + 21 tests slot maths, capacity race, hold expiry
4	Cart & checkout	Choose outlet, day and time; server recomputes the price; unpaid orders hold the slot for 15 minutes, then release it.	Tests + browser-verified
5	Paystack payments	Hosted checkout (card, bank transfer, USSD) — card data never touches our code. Webhook verifies HMAC-SHA512, amount, currency, reference; idempotent. App can reconcile via server → Paystack verify.	14 webhook tests live Paystack not yet exercised
6	Order tracking + push	Placed → Preparing → Ready → Collected, polled in-app and pushed via Expo/FCM at every change.	5 state tests push not yet on a device
7	QR pickup	Unforgeable per-order QR (HMAC). Staff scan; only a paid, READY order at that outlet can be collected — once. Order-code fallback.	Tests: forged, unpaid, double-scan, wrong outlet, race
8	Complaints & refunds	Report a wrong order with a photo; admin approves (full or partial) → Paystack refund to the original method → webhook finalises. Failed refunds reopen the complaint.	7 tests live refund not yet exercised
9	Staff & admin dashboard	Live order board (with sound), QR scanner, menu / outlets / staff management, complaints & refunds, sales reports.	Browser-driven against the live API

Orders are pickup-only at launch. Delivery, promo codes and loyalty are deliberately out of scope (see roadmap).

Colour — taken from the VeeJay logo



VeeJay Green
#019947
Logo green ·
login, accents



Deep Green
#017A39
Buttons & text
(AA contrast)



VeeJay Red
#F81A08
Logo red ·
highlights



Deep Red
#C8140A
Red text on white



Warm White
#FFFAF5
Canvas

Green and red are sampled from the logo itself. The pure logo shades sit at about 3.6:1 and 4:1 against white, so text and button fills use the slightly deeper variants, which clear WCAG AA ($\approx 5.4:1$ and $\approx 5.9:1$).

Principles

- **Food first** — large images, price always in green, “Add” is the loudest thing on a card.
- **State is always visible** — a four-step timeline with a red ring on “now”; one status pill per order.
- **Calm money screens** — one total, one button, a plain sentence about Paystack.
- **Big targets** — 50 px buttons, 34 px steppers, chips instead of dropdowns for slots.
- **Honest empty/error states** — tell people what happened and what to do next.

Components



Primary button · 14 px radius



Ghost button



14:00

Chips for day / time / category



Status pills

MEAL PLAN

VeeJay Meal Deal

#5,500

Combo card · red border



Preparing

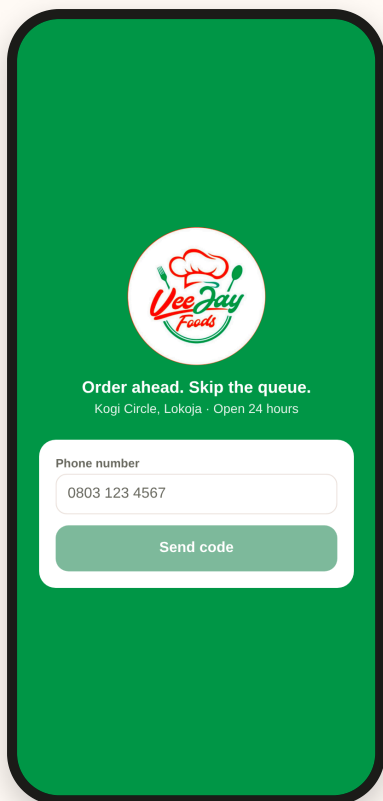
Ready

Timeline

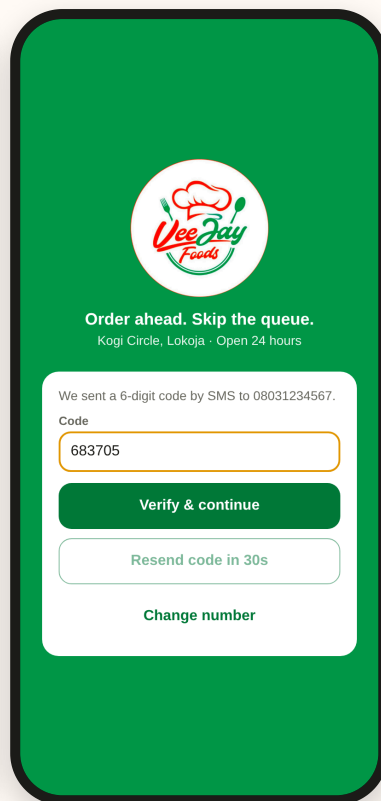
Type & spacing

System font stack (San Francisco / Roboto) for speed and native feel. Scale: 13 · 16 · 18 · 20 · 26 · 32. Spacing in 4 px steps; screens use a 16 px gutter; cards 14 px radius with a 1 px warm-grey border instead of shadows.

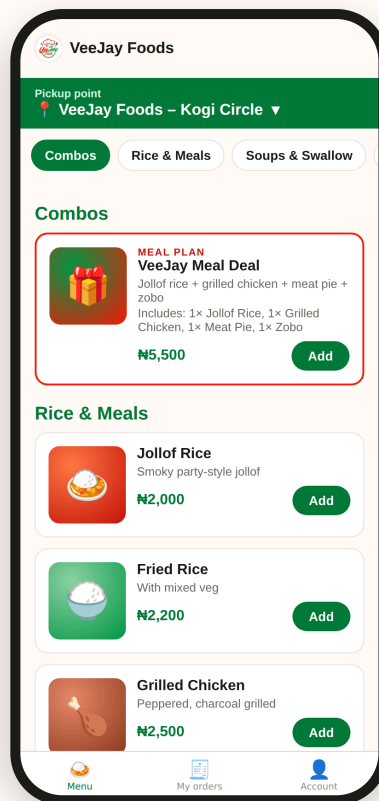
Brand name, logo and palette live in one theme file per surface (app, dashboard, API), so a future rebrand is a small change.

**Sign in**

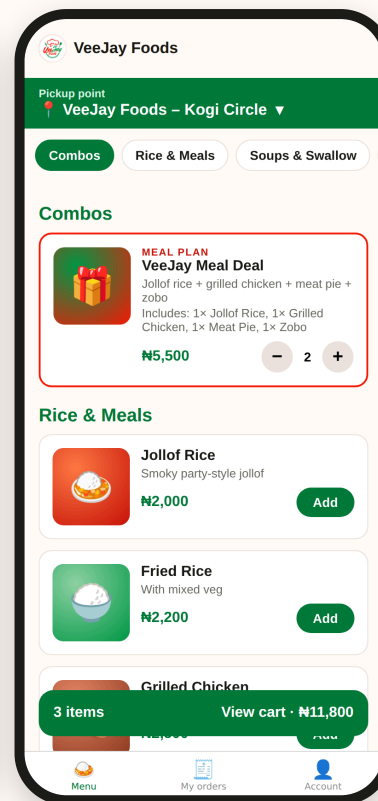
Logo, the Kogi Circle address and “Open 24 hours”. Phone number only — no passwords to forget.

**Verify**

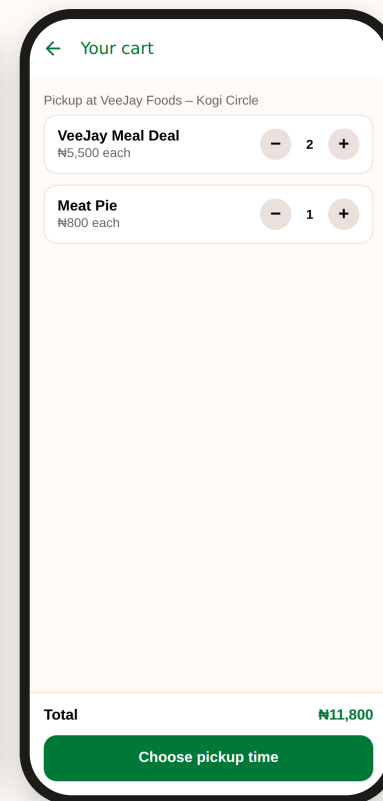
6-digit SMS code with a resend timer.

**Menu**

Pickup point on top; category chips; the meal plan is highlighted with a red border.

**Add to basket**

Steppers replace “Add”; a floating bar shows count and total.

**Cart**

Edit quantities; one clear next step.

← Pickup & pay

Pickup point
VeeJay Foods – Kogi Circle
Kogi Circle, along Ava Hotel, Ali Attah Road

Pickup day
Today Tomorrow Wed, 7 Oct Thu, 8 Oct

Pickup time

0:00	0:30	1:00	1:30	2:00
2:30	3:00	3:30	4:00	4:30
5:00	5:30	6:00	6:30	7:00
7:30	8:00	8:30	9:00	9:30
10:00	10:30	11:00	11:30	
12:00	12:30	13:00	13:30	
14:00	14:30	15:00	15:30	
16:00	16:30	17:00	17:30	
18:00	18:30	19:00	19:30	
20:00	20:30	21:00	21:30	
22:00	22:30	23:00	23:30	

Note for the kitchen (optional)
e.g. less pepper

2x VeeJay Meal Deal ₦11,000

Pickup & pay

Outlet, day chips, 30-minute time slots across the 24-hour day (full or past slots are hidden), kitchen note, itemised total.

What happens when “Pay” is tapped**1 Order is created on the server**

Prices are re-read from the database (the client can't set them). The chosen slot is locked and re-checked for capacity. The order sits in `PENDING_PAYMENT` for 15 minutes.

2 Paystack hosted checkout opens

In a secure in-app browser, with card, bank transfer and USSD enabled. Card numbers are entered on Paystack's page — never in our app or API.

3 Paystack calls our webhook

We verify the signature over the raw body, then check reference, amount and currency before moving the order to `PLACED`.

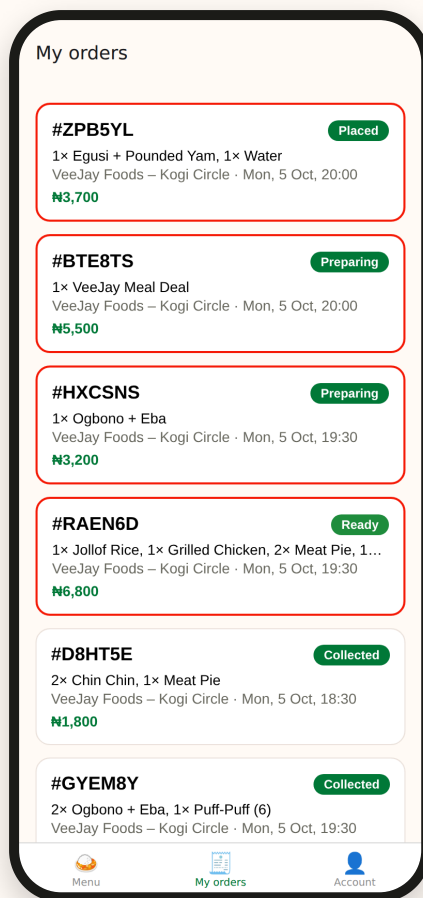
4 The app confirms with the server

On return it asks `/orders/:id/verify`, which asks Paystack directly. Webhook and verify share one code path, so they can never disagree.

Edge cases handled

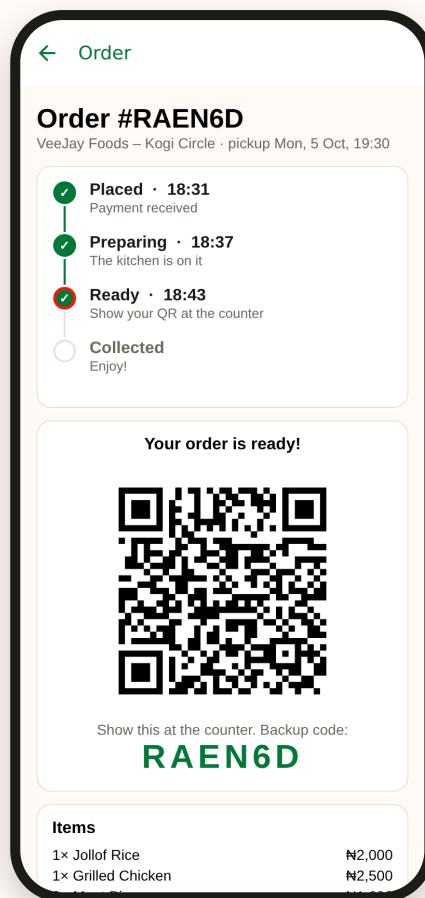
- **Webhook arrives twice (or six times at once)** → applied once, one push notification.
- **Amount or currency differs** → order stays unpaid; payment flagged.
- **Payment lands after the hold expired** → automatically refunded, order stays cancelled.
- **Server error mid-webhook** → returns 500 so Paystack retries; nothing half-applied.
- **Closed browser / lost signal** → order screen keeps checking and offers “Pay now” until the hold lapses.

Always know where your order is — and prove it's yours



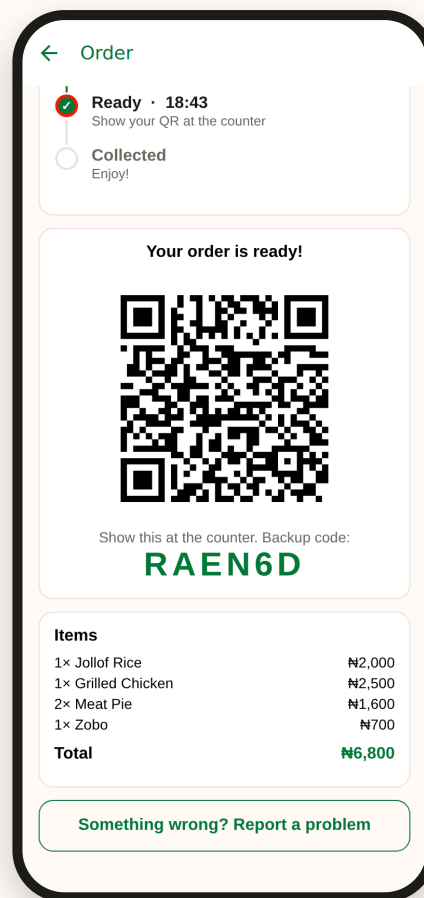
My orders

Active orders float to the top with a red outline; auto-refreshes.



Ready — show this QR

Four-step timeline with times; a large backup code if the camera fails.



Receipt

Snapshot of items and prices at the time of ordering, plus “Report a problem”.

Notifications

A push is sent on every status change:

- ✓ **Order placed** — payment received
- 🔍 **Preparing** — kitchen started
- 📦 **Ready for pickup** — show your QR
- 😊 **Collected** — thank you

Tapping a notification opens that order. Delivery via Expo Push (FCM on Android, APNs on iOS). Needs a development / production build, not Expo Go.

QR design

Payload `nbg1.<orderId>.<HMAC>`
— derived, not stored, so it can always be re-rendered and can't be guessed.

Report a problem

Reason chips, free-text details, camera or gallery photo.

Account

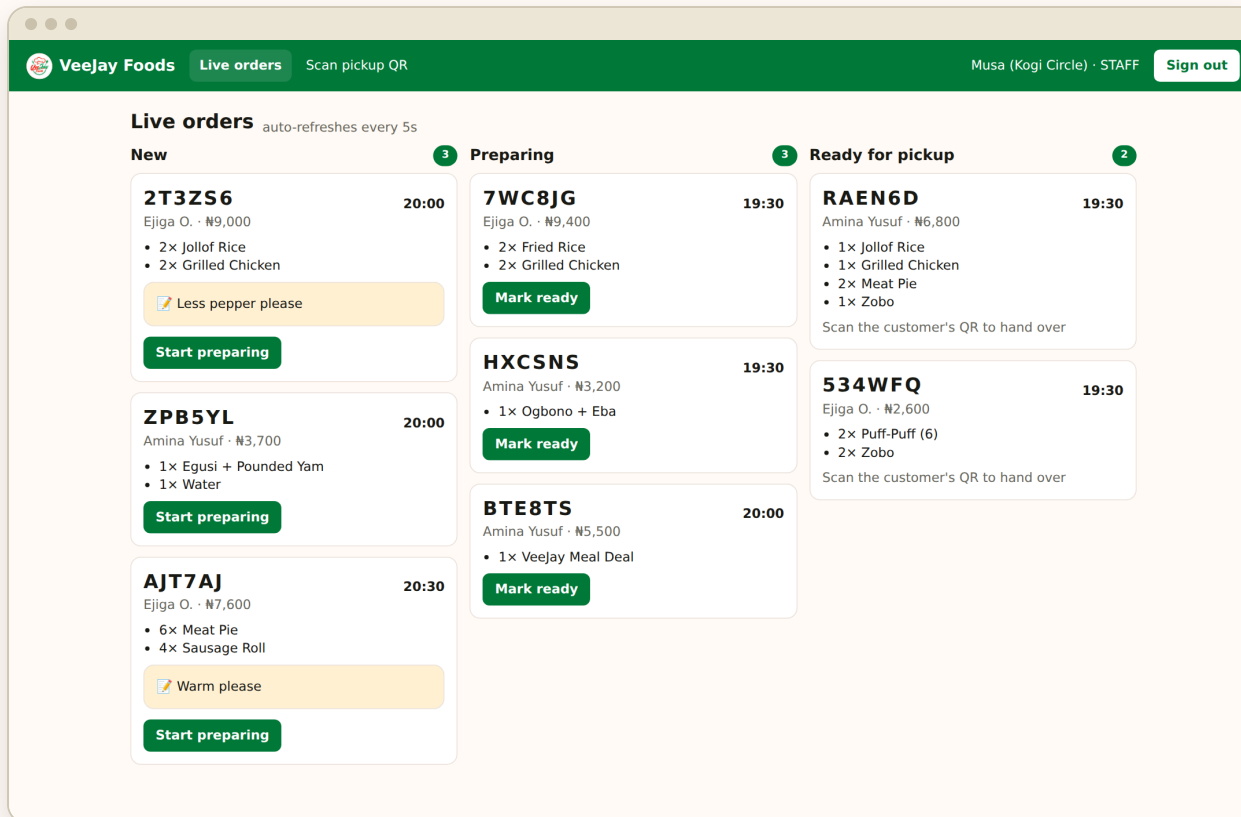
Name, phone, sign-out (with a warning if an order is in progress).

The refund journey

1. **Customer** reports a paid order — one open report per order; photo uploads are limited to JPG / PNG / WebP, 5 MB.
2. **Admin** sees it in *Complaints & refunds* with the photo and what was ordered (see next pages).
3. **Approve** (full or partial) → refund is sent to Paystack against the **original transaction**, so money returns to the original card / account. Amount is capped at what was paid.
4. **Paystack confirms** asynchronously → our webhook marks the refund **SUCCESS** and the payment **REFUNDED**.
5. **If Paystack refuses** → the complaint reopens automatically so an admin can retry. Double-clicks can't double-refund.

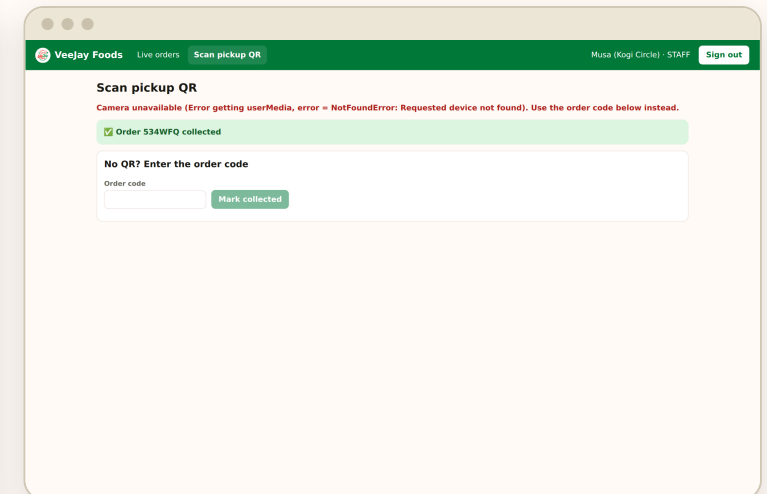
Account safety

Signing out is a deliberate choice only. Sessions end involuntarily only after 30 days of inactivity or a detected token-theft pattern — and never while an order is active.



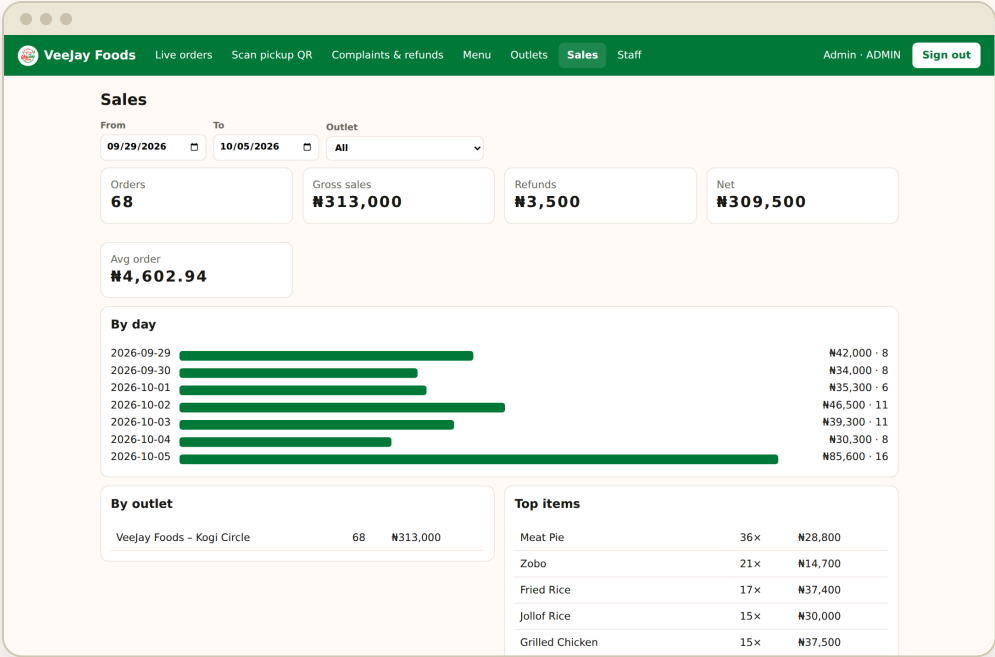
Live orders

Columns New → Preparing → Ready, sorted by pickup time; customer notes highlighted; refreshes every 5 s with a chime on new paid orders. Staff only see their own outlet.

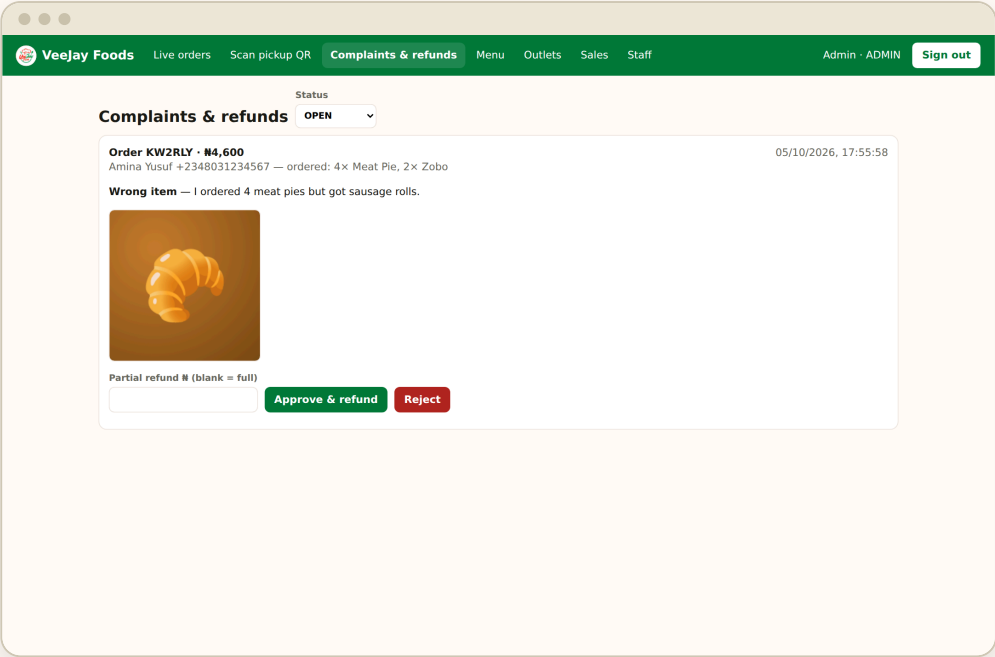


Scan pickup QR

Uses the device camera; a green banner confirms the hand-over. Typing the order code is the fallback (shown here after a manual hand-over). Double scans and not-ready orders are rejected with a clear message.



Sales
Date range and outlet filters; gross, refunds, net and average order; by-day bars; top items. Revenue counts paid orders only, minus successful refunds. (Sample data.)



Run the menu and the outlet without calling a developer

Menu

New category Add category

Combos + Add item

	VeeJay Meal Deal COMBO	Jollof rice + grilled chicken + meat pie + zobo	N5,500	☒ VeeJay	<button>Edit</button> <button>Hide</button>
--	--	---	--------	--	---

Rice & Meals + Add item

	Jollof Rice	Smoky party-style jollof	N2,000	☒ VeeJay	<button>Edit</button> <button>Hide</button>
	Fried Rice	With mixed veg	N2,200	☒ VeeJay	<button>Edit</button> <button>Hide</button>
	Grilled Chicken	Peppered, charcoal grilled	N2,500	☒ VeeJay	<button>Edit</button> <button>Hide</button>

Soups & Swallow + Add item

	Egusi + Pounded Yam	With assorted meat	N3,500	☒ VeeJay	<button>Edit</button> <button>Hide</button>
	Ogbono + Eba	With assorted meat	N3,200	☒ VeeJay	<button>Edit</button> <button>Hide</button>

Menu

Categories, items, combos / meal plans, images, prices, and a per-outlet availability switch to mark a dish sold out the moment the kitchen runs out. Items are hidden, never deleted, so past orders stay intact.

Pickup points

Name Address City ☒ Open for orders

Prep time (min) Slot length (min) Orders per slot

Sun	☒ Open	☒ 24 hours
Mon	☒ Open	☒ 24 hours
Tue	☒ Open	☒ 24 hours
Wed	☒ Open	☒ 24 hours
Thu	☒ Open	☒ 24 hours
Fri	☒ Open	☒ 24 hours
Sat	☒ Open	☒ 24 hours

Save VeeJay Foods - Kogi Circle

Add a pickup point

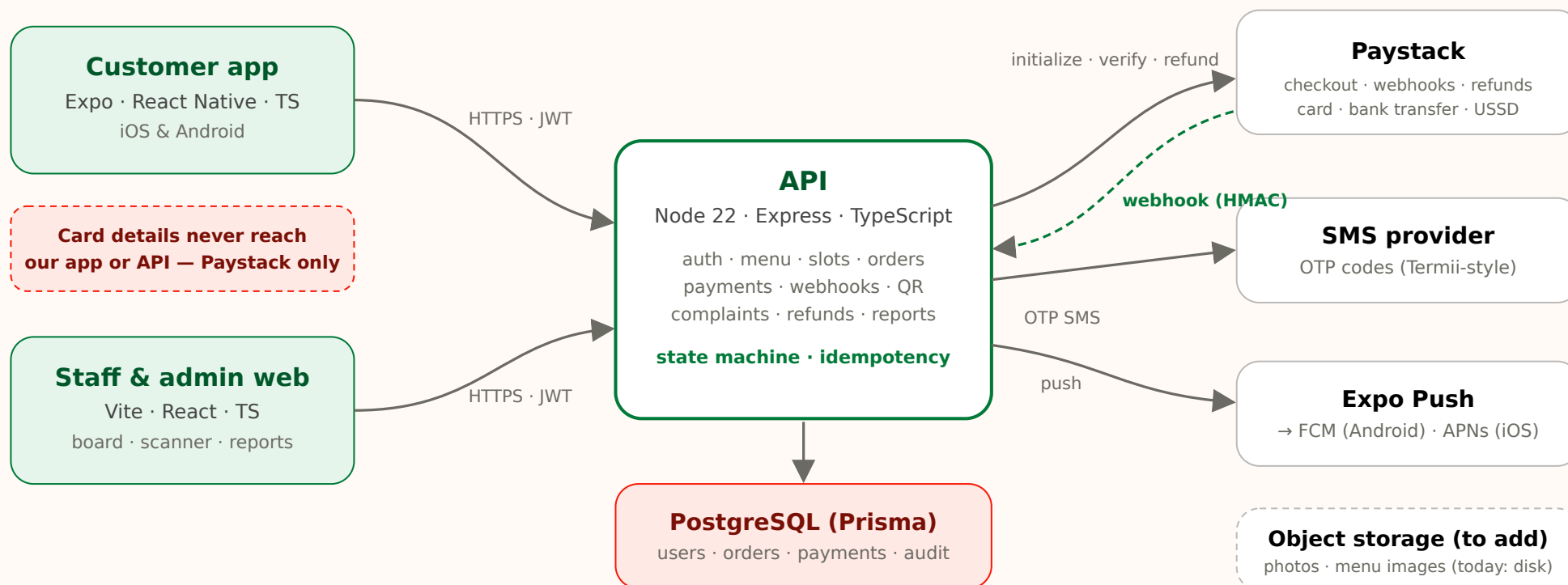
Name Address City Add

Outlets

Per-day hours with a “24 hours” switch (Kogi Circle is open round the clock), prep time, slot length and orders-per-slot capacity. Add a new branch in one form when VeeJay Foods grows.

Staff access

A seventh tab, **Staff**, adds or removes team members by phone number, role (Staff or Admin) and outlet. They sign in with an SMS code like customers do, and removing access takes effect immediately — the server re-checks the role on every request.



Why this stack

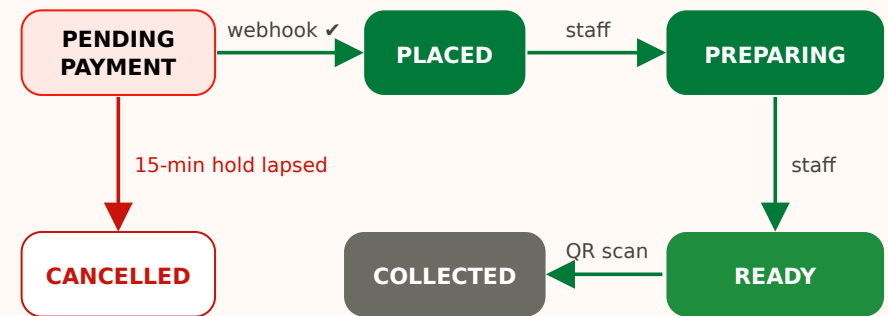
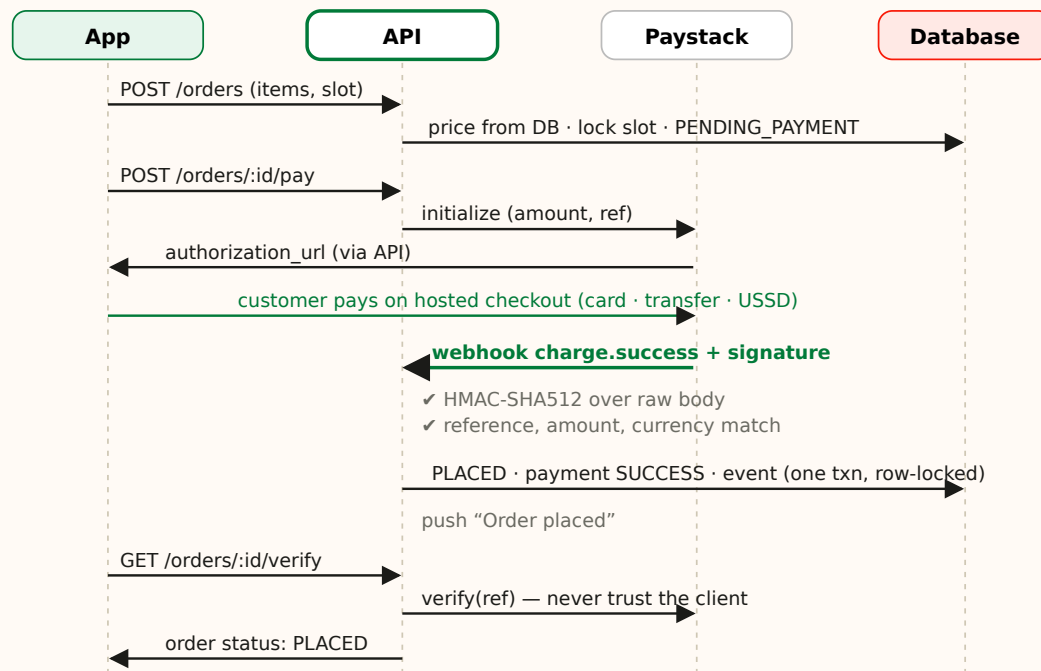
One language (TypeScript) across API, app and dashboard; Expo gives iOS + Android from one codebase and managed push; PostgreSQL gives transactions and row locks that money and slot-capacity need.

Seams for change

Paystack, SMS, push and storage are interfaces injected into services — swapped for fakes in tests and replaceable (e.g. Flutterwave, another SMS gateway, S3) without touching business logic.

Scale

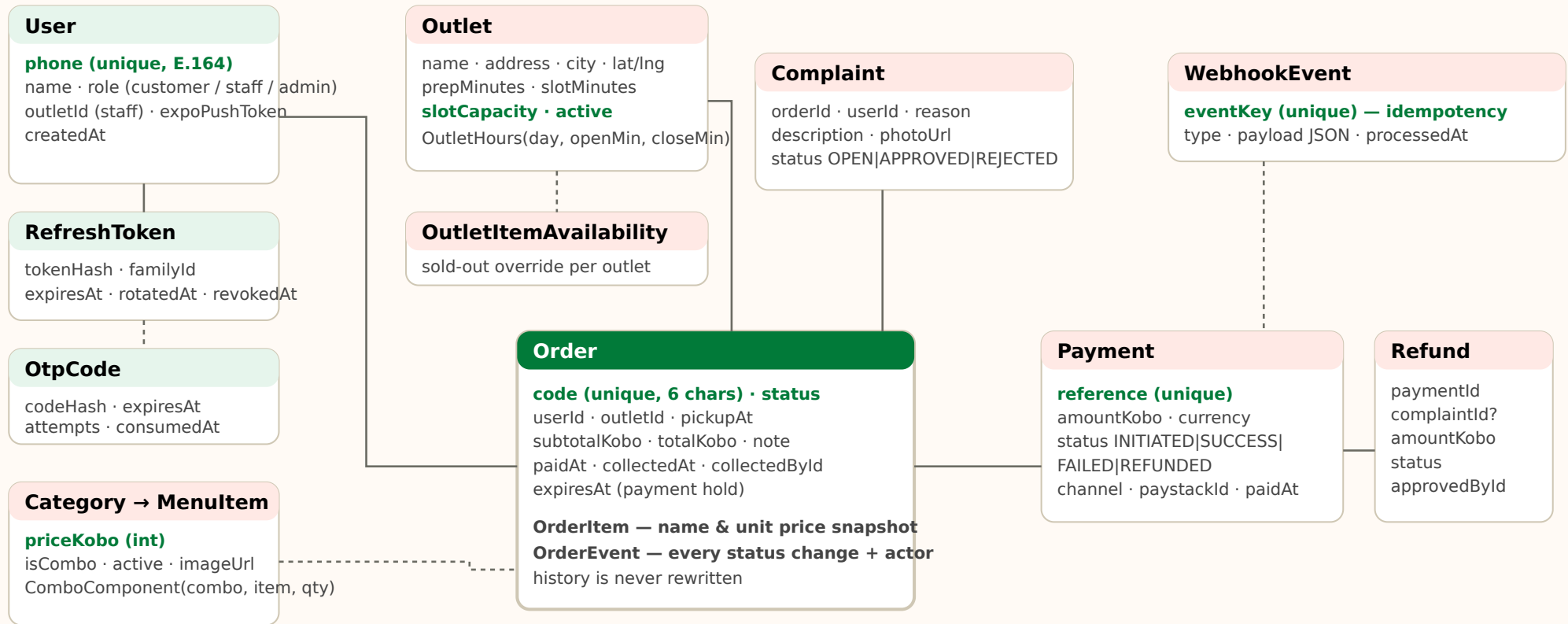
A single small Node instance and a managed Postgres comfortably serve a multi-outlet restaurant brand. The 60-second expiry job and 5-second polling are the only periodic load.



Green arrows are the only legal moves. Anything else is rejected with HTTP 409.

Rules enforced in code

- No skipping or going backwards; terminal states are final.
- A paid order can never be cancelled — only refunded.
- Each transition is a guarded `UPDATE ... WHERE status = from`, so two simultaneous taps apply once.
- Staff can set Preparing / Ready; **Collected** only via the QR scan (or the order-code fallback).

**Integrity by design**

Order lines snapshot name and price; payments are append-only with a unique Paystack reference; every status change writes an `OrderEvent`.

Idempotency

`WebhookEvent.eventKey` is unique, so a re-delivered webhook is detected at the database, even across concurrent requests.

Time zone

All instants are UTC; opening hours and slots are computed in Africa/Lagos (UTC+1, no DST) in one module.

Designed for the ways money apps actually fail

RISK	MITIGATION IN THE BUILD	EVIDENCE
Forged “payment succeeded” call	Webhook signature (HMAC-SHA512 over the <i>raw</i> body, constant-time compare); client never decides payment; verify goes server → Paystack.	Tests: missing / wrong signature, re-serialised body
Wrong amount or currency	Amount, currency and reference must match our own Payment row; mismatch marks payment failed and leaves the order unpaid.	Tests: tampered amount, USD
Duplicate / concurrent webhooks	Unique event key + row lock on the payment; single transaction applies the transition once.	Test: 6 simultaneous deliveries → 1 event, 1 push
Client edits the price	Server re-reads prices; request price fields are ignored.	Test with <code>priceKobo: 1</code> in the body
Over-booked pickup slots	Advisory lock per outlet+slot; unpaid holds expire after 15 minutes.	Test: 5 concurrent bookings, capacity 2 → 2 succeed
OTP brute force / abuse	Hashed codes, 5-minute expiry, 5 attempts, 3 codes per 10 min per phone, IP rate limit; code consumed atomically.	Tests: lock-out, reuse, expiry, rate limit
Stolen refresh token	Rotation with family revocation on replay (60 s grace for lost responses). Exception: never revoked while the user has an active order.	Tests: replay, grace, active-order cases
Fake or reused QR	HMAC-signed payload; paid + READY + same outlet required; atomic single use.	Tests: forged, tampered, double-scan race
Staff privilege drift	Role and outlet re-read from the database on every staff request, so removing access is immediate.	Code path + role tests
Double refund / over-refund	Complaint claimed with a conditional update; refund capped at the remaining paid amount; failed provider call reopens the complaint.	Tests: double-approve, cap, provider failure
Malicious uploads	Image-only MIME filter, 5 MB cap, random filenames.	Tests: HTML upload, 6 MB upload

67

backend tests, all passing, on a real PostgreSQL

21/21

Expo project health checks

0

uncaught script errors across the scripted app & dashboard sessions

Backend tests by area

Orders, slots, staff transitions, QR pickup	21
Sessions & OTP login	16
Paystack webhook & payment verify	14
Complaints, refunds, sales report	7
Order state machine	5
Slot calculation	4

Plus 3 app helper tests, a CI workflow that runs backend tests and typechecks / builds both clients on every push.

Verified

- Backend logic and money paths via automated tests (fakes for Paystack, SMS, push).
- Customer app (re-checked after the Veejay rebrand): type-checks, bundles for Android, and the full flow (sign-in → menu → cart → checkout → orders → QR → report) was driven in a phone-sized browser against the live API.
- Dashboard: builds; staff and admin flows driven in a browser against the live API.
- Real renders caught a bug that typecheck and the bundler missed (menu category chips collapsed) — fixed. Building the 24-hour outlet also exposed a hours-editor gap (24:00 can't be typed) — fixed with a "24 hours" switch.

Not yet verified — must happen before launch

- **Real Paystack** test-mode payment, webhook delivery and refund end-to-end.
- **Real SMS** delivery on Nigerian networks (and sender-ID registration).
- **Native devices** — iOS and Android: layout, keyboard handling, camera / photo picker, deep-link return from checkout.
- **Push notifications** on a real device build.
- Live camera QR scanning on the actual counter hardware.
- Load behaviour at peak (expected to be comfortable; not measured).
- Accessibility pass with a screen reader; low-end Android performance.

	W1	W2	W3	W4	W5	W6	W7	W8	W9	W10	W11	W12	W13	W14
0 · Build done	Backend · customer app · staff & admin dashboard · tests · CI (this repository)													
1 · Pilot readiness	Keys · SMS · builds · QA													
2 · Soft launch					Invite-only · training									
3 · Public launch									Kogi Circle · store release					
4 · Growth									Iterate →					

Indicative estimate assuming one full-stack developer plus part-time design and operations support. SMS sender-ID registration and App Store / Play review have external lead times and are the likeliest schedule risks.

1 · Pilot readiness (≈3 wks)

- Paystack live keys; webhook URL; test-mode drills
- SMS provider + sender ID
- EAS builds → TestFlight / Play internal testing
- Object storage for images; Sentry; DB backups
- Real menu, prices & food photography; privacy policy / NDPA notice

2 · Soft launch (≈3 wks)

- Kogi Circle only, invited customers
- Counter device + scan test; day & night staff training
- Daily review of orders, refunds, support
- Tune slot length / capacity / prep time

3 · Launch (≈4 wks)

- Store listings, screenshots, review buffer
- Launch offer + in-store QR posters
- Share on Instagram / Facebook
- Support playbook and on-call rota

4 · Growth (after)

- Promo codes, loyalty / reorder shortcuts
- Saved payment methods (Paystack tokens)
- Delivery option; more branches; orders > 3 days ahead
- Analytics, A/B tests, Pidgin / Yoruba / Hausa UI

Top risks

SMS deliverability

OTP is the front door. Sender IDs need registration; carriers filter.

Store review delays

Payments + login can trigger extra questions. Keep a 1-week buffer.

Staff adoption at peak

Needs a counter device, training, and the code-entry fallback.

Slot tuning

Wrong capacity either starves sales or recreates the queue.

24-hour operation

Night staffing and security; decide whether online orders run all night.

Image storage

Photos are on local disk today; lost on redeploy until moved to object storage.

Decisions needed

1. Confirm logo files and brand colours (sampled from the logo you shared).
2. Prep times and slot capacity per hour of day (day vs night).
3. Do online orders run 24 hours, or a shorter window at first?
4. Real menu, photography and prices (current ones are samples).
5. Refund policy: who may approve, partial refund rules, time limit.
6. Pickup-only at launch, or delivery in phase 4?
7. Customer support line: +234 906 685 9066 / +234 906 993 9673?
8. Paystack account owner and settlement bank.
9. Hosting region and provider (managed Postgres).

Go-live checklist

- ☐ Production secrets set (server refuses dev defaults)
- ☐ `PAYSTACK_SECRET_KEY` + webhook URL configured
- ☐ SMS provider key + sender ID
- ☐ Object storage for uploads
- ☐ CORS restricted to the dashboard origin
- ☐ Dashboard on HTTPS; API behind TLS
- ☐ Bundle IDs, icons, splash, store listing
- ☐ EAS project ID for push notifications
- ☐ Backups + monitoring + alerting
- ☐ Privacy policy / NDPA notice
- ☐ One real payment + one real refund, end to end

Repository layout

`veejay-foods/backend` API, Prisma schema, 67 tests

`veejay-foods/app` Expo customer app

`veejay-foods/admin` staff & admin dashboard

`veejay-foods/docs` architecture + this document